Cloud Computing - Assignment 1

Ivar van der Spoel (s3167445) & Ivor Brouwer(s2041685)

April 2025

1 Introduction

This is the report for assignment 1 of the Cloud Computing course. For the assignment an Automatically Scaling Web Application has to be built. The application has to consist of two different load-balancer rules and two different metric based auto-scalers. To evaluate the system experiments are performed.

2 Development

The system consists of a load-balancer, implemented using Flask, which distributes incoming image watermarking requests across web application container using and the Functions Framework running in the Podman container engine. Each backend instance processes a request by adding a watermark to an image and returning the result. Two load-balancing strategies are supported: Round Robin (RR), which cycles evenly through available containers, and a size based load-balancer which distributes based on the input image and requested watermark. The system includes two metric-based auto-scalers: one scaling based on the requests per second, and one based on the total throughput based on total throughput of the requests. The architecture is designed to minimize cold-start delays by maintaining a minimum number of ready containers at all times. The diagram of the system can be seen in Figure 1.

The host VM runs Debian 12.9.0 with 4Gb RAM and 4 processors, this was chosen as the base operating system because it is lightweight, and easy to set up in a virtualized environment. Python was selected as the programming language because the provided base implementation for the watermarking function was already in Python, and we had prior experience working with Python web frameworks like Flask. For load testing, Locust was used since it using Python was a great fit with the rest of our stack. Threading was used to combine the load-balancer and auto-scaler into a load coordinator for simplicity, enabling the management of shared global state without the overhead of setting up a database.

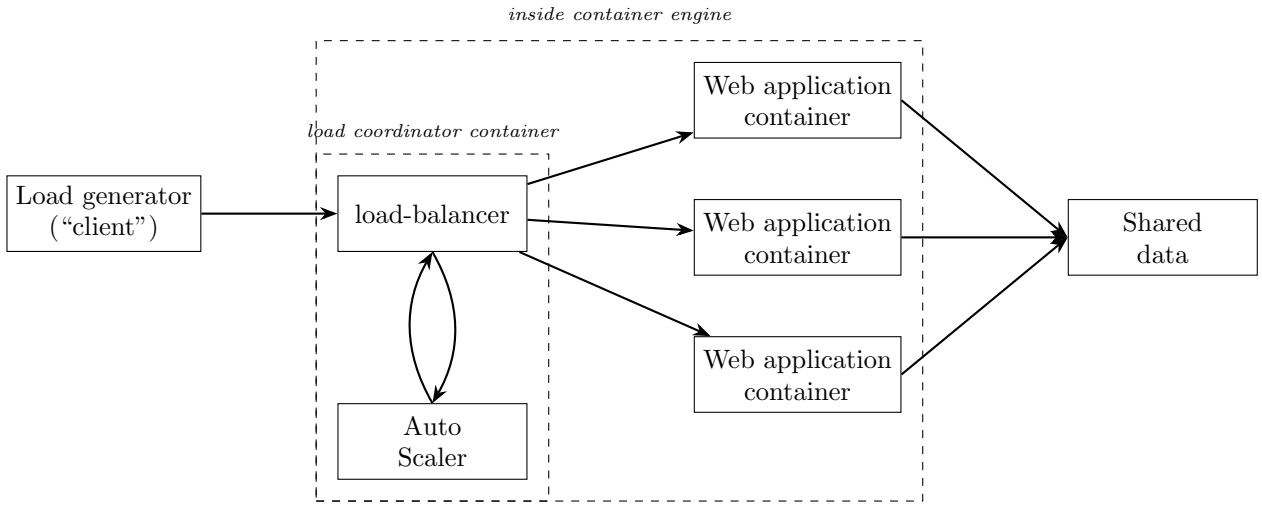


Figure 1: Diagram of the system, almost equivalent to the provided diagram by the professor but with a subtle difference that the auto-scaler in a combined container with the load-balancer.

3 Deployment

Deploying the full system can be done by using the provided container files, building the load-coordinator and the web-app. It is important that the `WATERMARK_DIR` environment variable is correctly set in the load-coordinator container-file since this is where the web-apps will access the watermark images as volumes and an absolute path is necessary to achieve this. Note that the

To start the load-balancer we need to mount multiple volumes and expose the port to the host, so to run we use the following command:

```
sudo podman run -d -p 8080:8080
-v /run/podman/podman.sock:/run/podman/podman.sock
--name load-coordinator
-v /path/to/materials-
assignment1/data/watermarks:/data/watermarks
--network cloud_network
load-coordinator
```

For the load generator, we use Locust. To install Locust, you can use pip:

```
pip install locust
```

Make sure to run it with the appropriate profile:

```
sudo LOCUST_PROFILE=small/medium/large/random locust --host http://127.0.0.1:8080/
```

4 Load Balancing

For load balancing we chose for Round Robin and an application-specific load-balancer based on the request size to see if balancing this would have a positive effect on the throughput.

4.1 Round Robin

The Round Robin load-balancer (RRLB) cycles through the list of available containers in a simple, fixed order. A global index variable is incremented after each request, ensuring the next container is selected in sequence. This approach assumes containers are equally capable and keeps the system simple and fair under steady load.

Algorithm 1 Round Robin Load Balancing (RRLB)

Require: List of running containers, global counter $i \leftarrow 0$
Ensure: Selects a container round-robin
  Select container $c \leftarrow \text{containers}[i \bmod |\text{containers}|]$
  $i \leftarrow i + 1$
  return c

4.2 Size Based

The Size-Based load-balancer (SBLB) routes requests depending on the estimated workload size. Each container keeps track of its current total workload, based on the input image size and the watermark size. The container with the least total assigned workload is selected for the new request, aiming to keep load distribution balanced even when individual requests vary significantly in size.

Algorithm 2 Size Based Load Balancing (SBLB)

Require: List of running containers
Ensure: Selects container with lowest workload
  Initialize $best_container \leftarrow$ `None`
  Initialize $min_workload \leftarrow +\infty$
  for all container c in containers **do**
    $w \leftarrow$ current workload of c
    if $w < min_workload$ **then**
      $min_workload \leftarrow w$
      $best_container \leftarrow c$
    end if
  end for
  return $best_container$

5 Auto Scaling

When we scale up we add the container to a set that the load-balancer can check for available containers, when scaling down we remove the container from this set so it won't receive new requests. When the container is done processing it is removed.

5.1 Request based

In the request-based auto-scaling, we monitor the request over a period of time. If the number of requests per second exceeds a threshold, a new container instance is started to handle the increased load, which reduces the average per container. This also works the other way around to scale down. This ensures the application can efficiently scale up during bursts of traffic and scale down when idle. We always keep a minimum amount of containers running.

Algorithm 3 Request-Based Auto-scaler

```
Require: Current containers
Ensure: Scaling down and up of web container apps
  while true do
      request_count ←calculated average requests per second
    if containers marked for removal then
        Remove idle container
    end if
    if (High request_count and below max containers) or below min containers
then
        Create new containers
    else if Low request_count and above min containers then
        Select container to remove
        Mark container for removal
    end if
    Sleep for 1 second
end while
```

5.2 Throughput based

For the throughput-based auto-scaling we use the total workload as the scaling metric, it is measured in megabytes per second (MB/s) over a period. The average of the workload over the containers is calculated, if this average exceeds a predefined threshold, a container is added resulting in a decrease in the load per container. If the throughput falls below a threshold and the pool of containers is above the minimum allowed size, the system scales down by stopping a container.

Algorithm 4 Throughput-Based Auto-scaler

Require: Current containers, measured container workloads
Ensure: Scaling down and up of web container apps based on throughput
  while true **do**
    workload_per_container ←calculated workload per container in MB/s
  if containers marked for removal **then**
    Remove idle container
  end if
  if High workload_per_container and below max containers or Below min containers **then**
    Create new containers
  else if Low workload_per_container and above min containers **then**
    Select container to remove
    Mark container for removal
  end if
  Sleep for 1 second
end while

5.3 Load Generator

As mentioned before we use locust for the load generator.

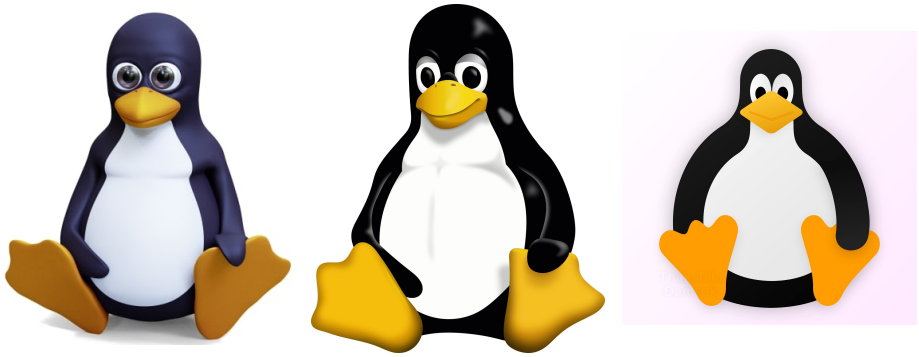


Figure 2: (a) 370x420(15.8KB) Figure 3: (b) 1200x1422(47.5KB) Figure 4: 2000x2000(63.1KB)

Figure 5: Three Tux images: small, medium and large.

The three different images used by the load generator are observed in Figure 5.

The goal of the load generator is to simulate users sending watermark HTTP requests to the web service, more specifically the load-balancer, under varying conditions. The load generator is configurable with `LOCUST_PROFILE` environment variable which allows for control over which image and watermark sizes are

selected, or 'random', which selects a random combination. Each user submits a POST request containing the image file and watermark size. For the final experiment a staged load shape is used.

6 Calibration Experiments

To test our container we limited the CPU count to 0.5, to accurately model scaling, as not limiting just divides the resources by the amount of containers.

6.1 Saturation point container

To find the saturation of a single container we send requests containing the different images in Figure 5 and each different watermark size. Running this experiment on a single container with both the small image and small watermark resulted in the graphs in Figure 6.

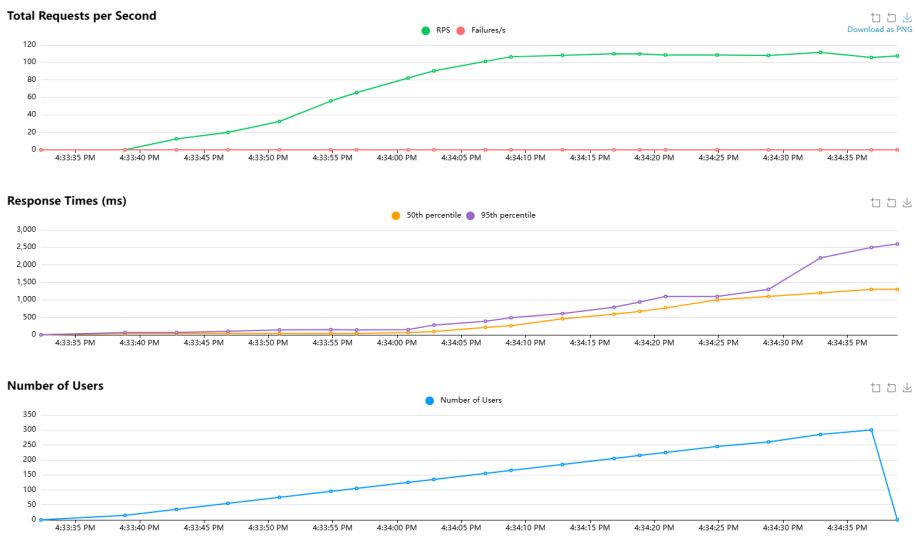


Figure 6: Graphs showing the saturation point of the graph.

This clearly shows that the container saturates at around 110 RPS. Testing with larger watermarks on this image did not result in slow down as the watermarks are too big for the image, causing the process to stop early. Using larger images as input did result in a significant reduction in RPS. Below in Figure 1 with the different combinations and the RPS and latency when first reaching this cap.

From these results we can see a clear correlation between the increasing image sizes and the earlier stagnation of the throughput, the latency significantly picks up the larger the images are as the containers will need more time to

Input Image	Watermark Size	Saturated RPS	50th% Latency(ms)	95th% Latency(ms)
Small	Small	110	500	720
Medium	Small	50	1000	1600
Medium	Medium	60	1000	1400
Medium	Large	53	1100	1600
Large	Small	33	1600	2300
Large	Medium	25.1	2000	3000
Large	Large	29	2100	2800

Table 1: Maximum RPS and latency for different image and watermark sizes.

process the images as the container is saturated and clients will have to wait longer until they get a response. We can see some weird occurrences of the RPS becoming higher in some cases and slowing down when applying the Medium watermark. We expected this could be due to the image not fully overlapping the input image, it will not process every pixel, which could in some cases cause speed ups.

6.2 Saturation point of multiple containers

To measure the saturation point of multiple containers, we observe the latency of Small, Medium and Large with the found set of users which causes saturation with 1 container as seen in the previous section.

The load balancing is done using our simple Round Robin load-balancer.

Input Image Size	1 Containers	2 Containers	4 Containers	8 Containers
50% Latency (ms)				
Small (110 users)	150	110	150	110
Medium (60 users)	230	96	97	99
Large (30 users)	400	160	110	110

Table 2: Measured RPS and 50th percentile latency for different numbers of containers and input image sizes.

In Table 2, the latencies are presented, showing that as the number of containers increases, the saturation effect diminishes, and no bottleneck is observed. However, this does not hold true for smaller images, as the latency remains relatively constant. One potential way of showing a saturation would be to increase the number of users.

6.3 Required spawn time of containers

A web application container is required to preload the watermark images before it is ready to receive http requests and apply the watermark. By investigating the execution time of `preload_images()` the spawn time of a container is known.

It appears that it takes 0.04 seconds to preload the required images. After, the containers are ready to receive http requests. The container is set as "running" before images are preloaded, this is a limitation of Podman Python. To ensure that containers do not receive http requests, the auto-scaler waits 1 second before putting the container in a set of available containers for the load-balancer to select from.

7 Final Experiment

In this experiment, we explore two distinct approaches to auto-scaling: application-level and demand based. We developed two types of auto-scalers a Throughput Based Auto-Scaler (application-level) and a Request Based Auto-Scaler (demand based) to evaluate these approaches. To assess their effectiveness under different conditions, we also implemented two load balancing strategies: Round Robin and Request Size Based Balancing. Which raises the following research question:
RQ: *How do application-level and demand based auto-scaler, in combination with different load-balancers, perform in varying workloads?*

There are two SLAs we would like to adhere.

- Scaling Time: New containers should (de)spawn within 10 seconds when detecting a significant load change.
- Error Rate: 0% of HTTP requests may fail in the duration of the experiment.

7.1 Experimental setup

In this experiment, three different Tux images are randomly selected as inputs. Each selected image is watermarked using a randomly chosen watermark, with all odds being equally likely. Additionally, we log the container spawn times throughout the experiment. Each container has 0.5 CPU count.

For auto-scaling, there are certain variables to choose:

Request based used an average RPS of 10 over all containers to scale up, and 5 to scale down.

Throughput based used an MB/s of 1 to scale up, and 0.5 to scale down.

The Locust-based load test follows a predefined timeline to simulate varying load conditions as can be seen in Table 3.

Time	Ramp up	Max users
0-30	10	60
30-60	10	20
60-90	10	100
90-120	5	10
120 - 150	5	0

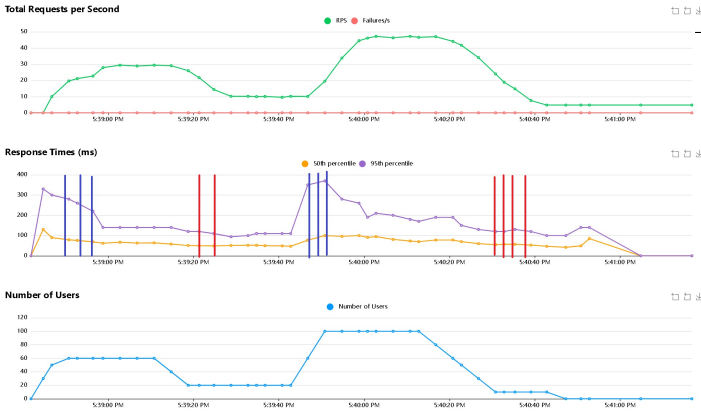
Table 3: Locust timeline. Ramp up is clients started or removed per second.

7.2 Evaluation

The timelines for each combination of auto-scaler and load-balancer are presented in Figures 7 to 10. Accompanying each figure is a table that lists the exact timestamps of container spawning and despawning events. Blue lines indicate a spawn and red indicates a despawn.

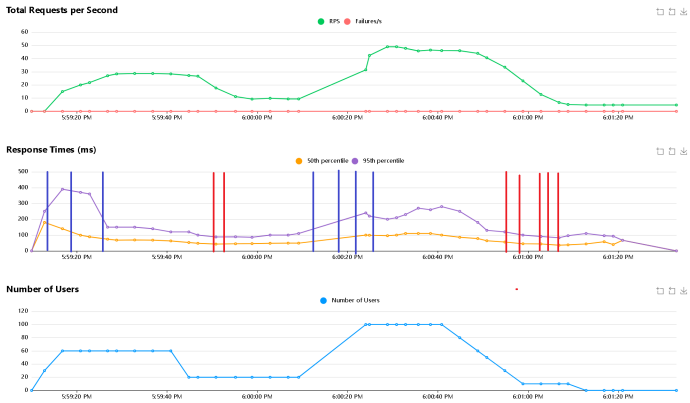
Fortunately, all SLAs are met: no request failures occur, and the system successfully scales up and down as expected.

It becomes apparent that all four configurations are suitable options for this web application. No severe latency bottlenecks are observed, only minor delays occur, which are quickly resolved through up-scaling. Furthermore, the user count closely tracks the RPS, indicating that requests are processed efficiently and fairly across all configurations.



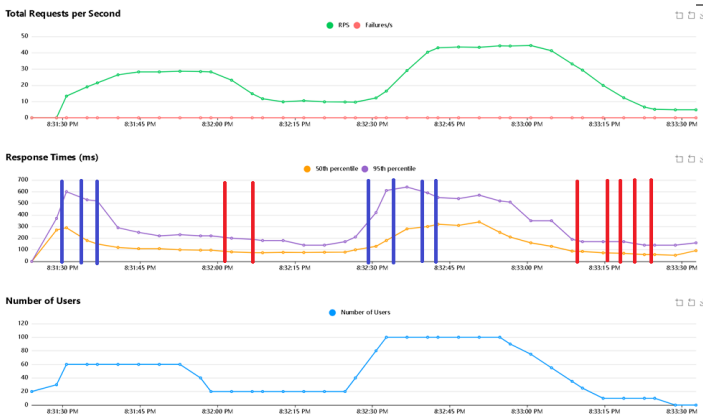
Event	Timestamp
Spawn	05:38:48 PM
Spawn	05:38:51 PM
Spawn	05:38:57 PM
Despawn	05:39:21 PM
Despawn	05:39:25 PM
Spawn	05:39:48 PM
Spawn	05:39:51 PM
Spawn	05:39:53 PM
Despawn	05:40:29 PM
Despawn	05:40:33 PM
Despawn	05:40:35 PM
Despawn	05:40:38 PM

Figure 7: Experiment results with Request Based auto-scaler and Size Based load-balancer.



Event	Timestamp
Spawn	05:59:16 PM
Spawn	05:59:19 PM
Spawn	05:59:26 PM
Despawn	05:59:50 PM
Despawn	05:59:54 PM
Spawn	06:00:16 PM
Spawn	06:00:19 PM
Spawn	06:00:21 PM
Spawn	06:00:24 PM
Despawn	06:00:57 PM
Despawn	06:00:59 PM
Despawn	06:01:02 PM
Despawn	06:01:05 PM
Despawn	06:01:08 PM

Figure 8: Experiment results with Request Based auto-scaler and Round Robin load-balancer.



Event	Timestamp
Spawn	08:31:30 PM
Spawn	08:31:34 PM
Spawn	08:31:37 PM
Despawn	08:32:04 PM
Despawn	08:32:08 PM
Spawn	08:32:29 PM
Spawn	08:32:33 PM
Spawn	08:32:36 PM
Spawn	08:32:39 PM
Despawn	08:33:11 PM
Despawn	08:33:16 PM
Despawn	08:33:18 PM
Despawn	08:33:21 PM
Despawn	08:33:24 PM

Figure 9: Experiment results with Throughput Based auto-scaler and Size Based load-balancer.

Event	Timestamp
Spawn	08:13:37 PM
Spawn	08:13:40 PM
Spawn	08:13:44 PM
Despawn	08:14:11 PM
Despawn	08:14:15 PM
Spawn	08:14:38 PM
Spawn	08:14:40 PM
Spawn	08:14:43 PM
Spawn	08:14:51 PM
Despawn	08:15:18 PM
Despawn	08:15:22 PM
Despawn	08:15:24 PM
Despawn	08:15:27 PM
Despawn	08:15:30 PM

Figure 10: Experiment results with Throughput Based auto-scaler and Round Robin load-balancer.

8 Contributions

Task	Contributors
System design	Ivor
Load-balancing	Ivor & Ivar
Auto-scaling	Ivor & Ivar
Podman container setup	Ivor
Experiments	Ivor & Ivar
Documentation and report writing	Ivor & Ivar

Table 4: Task distribution.

In Table 4 the task distribution is shown. Ivor has been involved in more varying tasks. However, Ivar has been occupied in other parts of the assignment.

9 Resources

During the completion of this assignment, we made use of the following resources:

- Podman Python client documentation — for container management and monitoring.
- Locust documentation — for load generation and testing of the system under stress.
- Discussions with Professor Rietveld — for implementation ideas for the load-balancer and scaling rules.

10 Conclusion

We successfully implemented an automatically scaling web application. Our system incorporates two types of load-balancers (Round Robin and Size-Based) and two types of metric-based auto-scalers (Request-Based and Throughput-Based). The experiments showed that our system successfully adheres to the defined SLAs: no HTTP request failures occurred, and container scaling actions completed within the required 10-second window. While performing the calibration experiments we found the saturation points of containers and showed that scaling could reduce the overall latency at these saturation points, improving performance.

Overall, this project deepened our understanding of cloud computing, particularly in container balancing and scaling. Future work could include running the containers in the cloud and deeper experimentation with actual scaling of hardware.